USING DEBUGGING TOOLS

Visual Basic 6.0 | OAVS Computer Teacher Exam - Study Notes

Syllabus order covered in these notes

A. Types of errors & debug menu (i Types of errors, ii Debug menu) B. Testing the application (i Immediate window, ii Using debug and local window, iii Setting watch expression) C. Implementing error handler (i How VB handles the runtime error, ii VB error handler) D. VB error handling options (i How VB handles the runtime error, ii Disabling the error handler) E. Inline error handling | then Quick Revision

A. TYPES OF ERRORS & DEBUG MENU

A (i) Types of errors

Definition & purpose. An **error (bug)** is any fault that stops a program from running or makes it give wrong results. **Debugging** means finding and removing errors. VB errors are classified into three types because each is detected at a different time and removed in a different way.

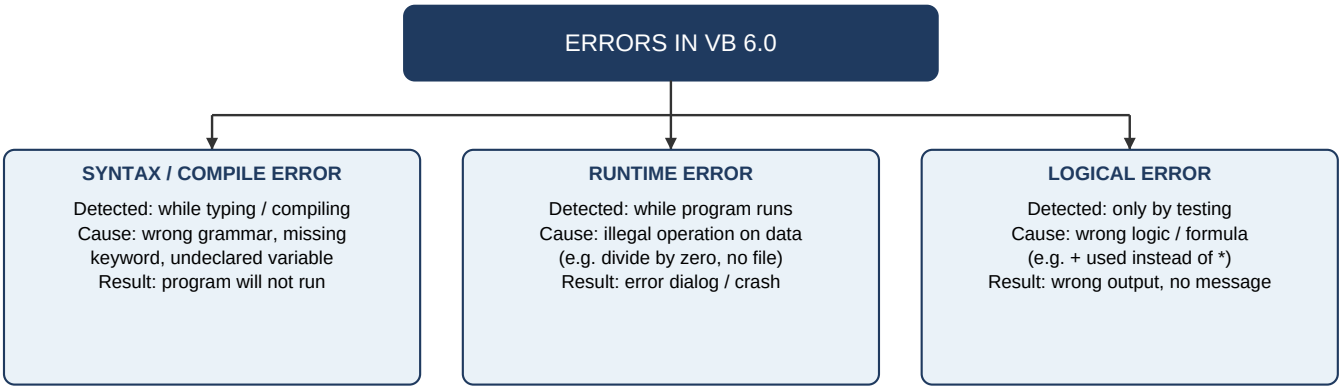

Fig. 1: Classification of errors in VB 6.0

Feature	Syntax / Compile error	Runtime error	Logical error
Detected	At design time (Auto Syntax Check) or when compiling/starting the program	While the program is running	Never by VB; found only by testing
VB message	Yes - 'Compile error: ...' and line highlighted	Yes - 'Run-time error n' dialog with End / Debug / Help	No message at all
Effect	Program cannot run	Program stops (or is trapped by handler)	Runs smoothly but result is wrong
Example	`If x > 5 MsgBox "Hi"` (Then missing); misspelt keyword	`x = 10 / 0`; opening a missing file	`Area = L + B` instead of `L * B`
Removed by	Correcting the code	Error handler + validation	Testing + debugging tools

Common runtime error numbers (used with `Err.Number`):

No.	Message	No.	Message
5	Invalid procedure call or argument	62	Input past end of file
6	Overflow	70	Permission denied
9	Subscript out of range	76	Path not found
11	Division by zero	91	Object variable or With block variable not set
13	Type mismatch	94	Invalid use of Null
53	File not found	71	Disk not ready

IMPORTANT FOR OAVS EXAM

- **Auto Syntax Check** is set under *Tools* > *Options* > *Editor*; it flags a syntax error as soon as the line is left (shown in red).
- **Option Explicit** (Require Variable Declaration) makes an undeclared variable a compile error: *'Variable not defined'*.
- *Run* > *Start With Full Compile* (Ctrl+F5) finds all compile errors before execution starts; a normal start (F5) may report them only when that code is reached.
- Division by zero (11), Type mismatch (13), Subscript out of range (9), File not found (53) are the most asked runtime errors.

COMMON MISTAKES / CONFUSING POINTS

- 'Type mismatch' and 'Division by zero' are **runtime**, not syntax or logical, errors.
- Without Option Explicit, a misspelt variable name silently creates a new empty Variant - the result is a **logical** error, not a syntax error.
- A program with a logical error never shows an error message - 'no message' does not mean 'no bug'.

SUMMARY

Syntax = grammar mistake (before run); Runtime = failure during run (error number); Logical = wrong result without any message.

A (ii) Debug menu

Definition & purpose. The **Debug menu** holds the commands used to run a program step by step, pause it, and examine it. (The same commands are on the *Debug toolbar*: *View* > *Toolbars* > *Debug*.) VB works in three modes shown in the title bar: **Design time** [design], **Run time** [run] and **Break mode** [break] - the paused state in which most debugging commands work.

Debug menu command	Shortcut	Purpose
Step Into	F8	Executes one statement at a time; enters called procedures
Step Over	Shift+F8	Runs a called procedure as one single step
Step Out	Ctrl+Shift+F8	Runs the rest of the current procedure; stops in the caller
Run To Cursor	Ctrl+F8	Runs normally until the line holding the cursor
Add Watch...	-	Creates a watch expression (see B iii)
Edit Watch...	Ctrl+W	Modifies or deletes an existing watch
Quick Watch	Shift+F9	One-time display of the value of a selected expression
Toggle Breakpoint	F9	Sets / removes a breakpoint on the current line
Clear All Breakpoints	Ctrl+Shift+F9	Removes every breakpoint
Set Next Statement	Ctrl+F9	Chooses the next line to run (same procedure only)
Show Next Statement	-	Moves cursor to the line that will run next

Ways to enter Break mode: a breakpoint (F9) is reached; *Run* > *Break* / **Ctrl+Break**; a `Stop` statement; `Debug.Assert` condition is False; a watch condition triggers; or **Debug** is clicked in a runtime error dialog. A breakpoint line is shown with a dark-red highlight and margin dot; the line about to run is shown in **yellow** with an arrow. Related: *View* > *Call Stack* (Ctrl+L) lists the procedures currently active.

IMPORTANT FOR OAVS EXAM

- Remember: **F8** Step Into, **Shift+F8** Step Over, **Ctrl+Shift+F8** Step Out, **F9** Breakpoint, **Shift+F9** Quick Watch, **Ctrl+F8** Run To Cursor.
- **Breakpoints and watch expressions are not saved** with the project; they are lost when the project is closed.
- Breakpoints are ignored in a compiled EXE.

COMMON MISTAKES / CONFUSING POINTS

- **Step Over** still runs the called procedure (without showing it); **Step Out** leaves the *current* procedure. Step Into goes *inside*.
- A breakpoint cannot be placed on comments or declaration lines (`Dim`) - they are not executable.
- **Set Next Statement** only works within the current procedure and skipped lines are NOT executed.

SUMMARY

The Debug menu controls execution (step, run to cursor, breakpoints) and inspection (watches). Most commands need Break mode.

B. TESTING THE APPLICATION

Testing means running the application with sample (normal and extreme) data and comparing the actual output with the expected output. When the result is wrong, the program is paused in Break mode and examined with the windows below.

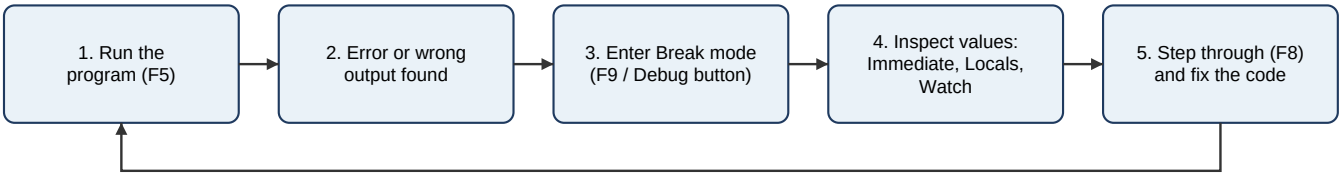

Fig. 2: Debugging / testing workflow

B (i) Immediate window

Definition & purpose. The **Immediate window** (*View > Immediate Window*, **Ctrl+G**) is an interactive window in which a single line of code is executed as soon as **Enter** is pressed. It is used to print values, test statements and functions, and change variables while testing. (In older VB versions it was called the *Debug window*.)

Use	How	Example
Display a value	`? expression` (? is short for Print)	`? 5 * 3` gives 15
Run a statement	Type the statement and press Enter	`Text1.Text = "Hi"`
Change a variable	Assign a new value in Break mode	`total = 0`
Call a procedure / function	Type its name or use ?	`? Square(4)`
Several statements	Separate with a colon (one line)	`a = 2 : ? a * a`
Print from code	`Debug.Print` writes to the window at run time	`Debug.Print "i="; i`

```
Private Sub Command1_Click()
    Dim i As Integer
    For i = 1 To 3
        Debug.Print "i = "; i      ' appears in Immediate window
    Next i
End Sub
```

OUTPUT

Immediate window after clicking Command1 (program above)

```
i =  1
i =  2
i =  3
```

Typed directly in the Immediate window (press Enter after each line)

```
? 5 * 3
 15
a = 2 : ? a * a
 4
```

Note

```
VB reserves a space for the sign of a number, so a space appears before each value.
```

IMPORTANT FOR OAVS EXAM

- **Debug.Print** output goes only to the Immediate window; it is **ignored in a compiled EXE** and does not stop the program (unlike MsgBox).
- `Debug.Assert condition` enters Break mode when the condition is **False**.
- Separators in Debug.Print: **;** prints items side by side; **,** moves to the next print zone.

COMMON MISTAKES / CONFUSING POINTS

- Typed lines are **not saved** in the project; Dim and multi-line blocks (If...End If) cannot be used - only single-line statements.
- Procedure-level variables are visible only when execution is paused *inside* that procedure.

SUMMARY

Immediate window = Ctrl+G; use ? to display and Debug.Print to log values; one executable line at a time.

B (ii) Using debug and local window

Definition & purpose. Debugging a running program means setting breakpoints, stepping through code and watching how variables change. The **Locals window** (*View > Locals Window*) automatically shows **all variables of the current procedure** so no expression has to be typed.

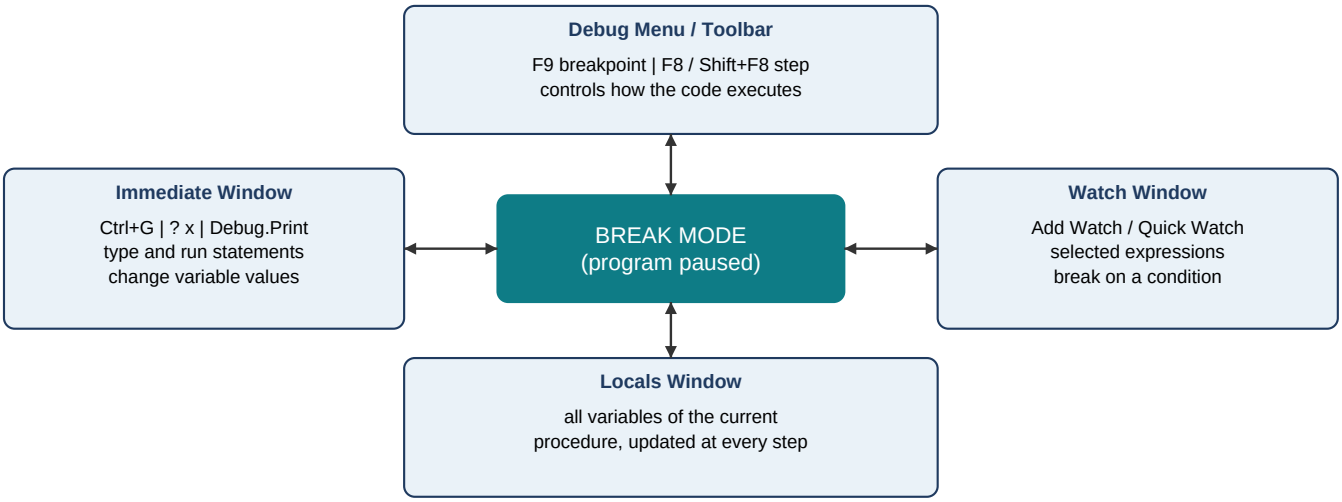

Fig. 3: Relationship between Debug menu, Immediate, Locals and Watch windows in Break mode

Locals window feature	Detail
Columns	**Expression** (variable name), **Value**, **Type**
Content	Variables of the current procedure plus module-level data under **Me** (form/class); arrays and objects expand with + sign
Updating	Refreshed automatically at every step; changes when execution enters another procedure
Editing	Click a Value cell, type a new value, press Enter - the running program uses it
Availability	Contents are shown only in Break mode

Working (debug session): set a breakpoint with F9, run (F5), program pauses; press F8 repeatedly and watch the Locals window.

```
Private Sub Command1_Click()
    Dim i As Integer, total As Integer
    For i = 1 To 3                ' <- F9 breakpoint here
        total = total + i
    Next i
    Debug.Print total
End Sub
```

After loop pass	Locals: i	Locals: total
1	1	1
2	2	3
3	3	6

OUTPUT

Locals window while paused at the Next i line of the last pass

```
Expression    Value     Type
i             3         Integer
total         6         Integer
```

Immediate window after the program finishes (Debug.Print total)

```
 6
```

IMPORTANT FOR OAVS EXAM

- Locals window shows **local** (procedure-level) variables only; use a **watch** to follow a variable across procedures.
- Hovering the mouse over a variable in Break mode shows its value (Auto Data Tips).

COMMON MISTAKES / CONFUSING POINTS

- Locals window = automatic, no typing; Watch window = only the expressions you choose; Immediate = you type commands.
- A line shown in yellow has **not yet executed**; its effect appears only after the next step.

SUMMARY

Break and step through code (F9, F8); the Locals window displays and lets you edit all current-procedure variables automatically.

B (iii) Setting watch expression

Definition & purpose. A **watch expression** is a variable, property or expression whose value VB monitors while the program runs. It can only display the value, or **pause the program** when a condition is met - useful for loops that run thousands of times.

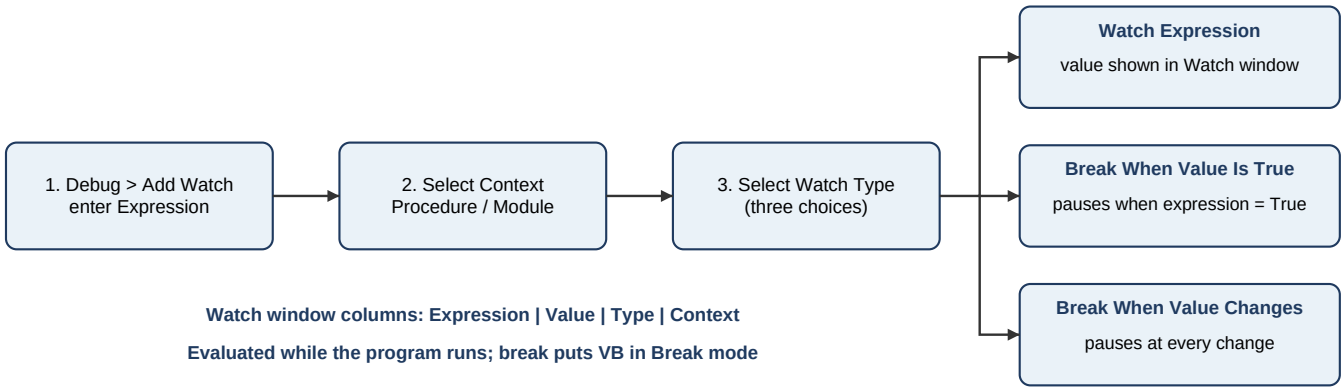

Fig. 4: Watch expression process (Add Watch dialog and watch types)

Add Watch dialog field	Meaning
Expression	Variable or expression to watch, e.g. `total` or `total > 100`
Context - Procedure / Module	Where the expression is evaluated (scope of the watch)
Watch Expression	Only displays the value in the Watch window
Break When Value Is True	Enters Break mode when the expression becomes **True** (non-zero)
Break When Value Changes	Enters Break mode every time the value **changes**

Working: *Debug* > *Add Watch* (or right-click > Add Watch), fill the dialog, OK. The **Watch window** (*View* > *Watch Window*) lists Expression, Value, Type and Context. **Edit Watch** (Ctrl+W) changes or deletes it. **Quick Watch** (Shift+F9) gives a temporary value with an *Add* button to make it a permanent watch.

```
' Watch:  total > 100     Type: Break When Value Is True
Dim i As Integer, total As Integer
For i = 1 To 1000
    total = total + i       ' program pauses as soon as total > 100
Next i
```

OUTPUT

Result

```
The program enters Break mode by itself on the line after the
statement that made the watch True (here after i = 14).
```

Watch window at that moment

```
Expression     Value   Type      Context
total > 100    True    Boolean   Command1_Click
```

Typed in the Immediate window

```
? i
 14
? total
 105
```

IMPORTANT FOR OAVS EXAM

- Three watch types: **Watch Expression**, **Break When Value Is True**, **Break When Value Changes**.
- Watch window columns: **Expression, Value, Type, Context**. Watches are **not saved** with the project.
- Breakpoint stops at a chosen *line*; a watch stops on a chosen *condition*.

COMMON MISTAKES / CONFUSING POINTS

- A watch with a narrow Context shows **<Out of context>** when execution is outside that procedure - it is not an error.
- Quick Watch is temporary (one look); Add Watch is permanent for the session.
- Too many watches slow program execution.

SUMMARY

Add Watch = expression + context + watch type; it monitors values or breaks on a True condition or a change.

C. IMPLEMENTING ERROR HANDLER

C (i) How VB handles the runtime error

Definition & purpose. A **runtime error** occurs when VB cannot carry out a statement. Whenever one occurs VB fills the **Err object** and then looks for an enabled error handler. Understanding this default behaviour shows why an error handler is needed.

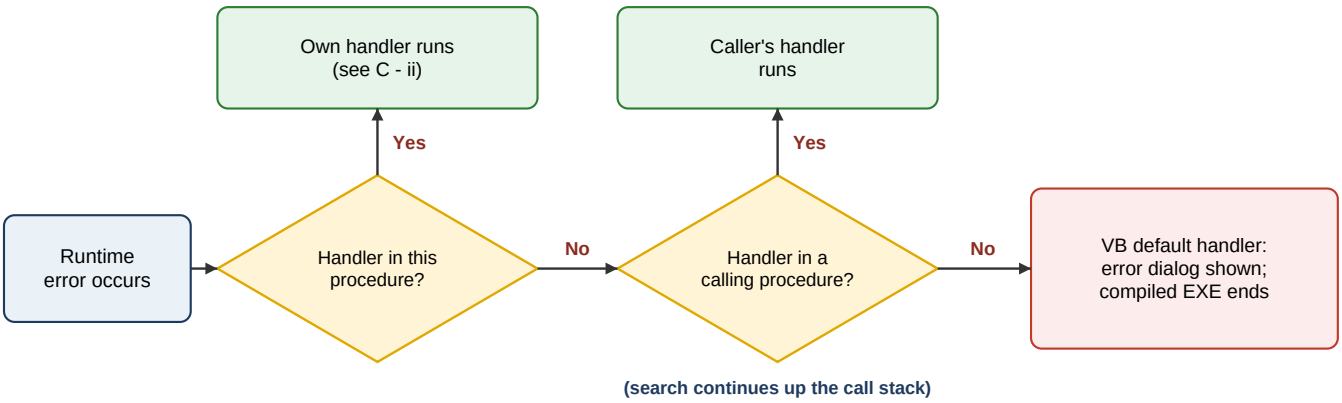

Fig. 5: Runtime error handling flow (default VB behaviour)

Situation	What VB does
No handler anywhere in the call stack - in the IDE	Shows **Run-time error 'n'** dialog with **Continue** (only for some errors), **End**, **Debug**, **Help**. Debug opens the code in Break mode with the failing line highlighted.
No handler - compiled EXE	Displays the error message and the program **terminates abruptly**; unsaved data is lost.
Handler found	Control jumps to the handler; program may continue (see C ii).

IMPORTANT FOR OAVS EXAM

- VB searches first the **current procedure**, then the **calling procedures up the call stack**; if none has an enabled handler, the default handler runs.
- In the dialog: **End** stops the program, **Debug** goes to the error line in Break mode.

COMMON MISTAKES / CONFUSING POINTS

- Without a handler, an unhandled error crashes the EXE - the default dialog is not a 'handler written by the programmer'.
- An error inside an *unprotected* procedure can still be caught by the *caller's* handler.

SUMMARY

Unhandled runtime error: search up the call stack, then default dialog; EXE terminates.

C (ii) VB error handler

Definition & purpose. An **error handler** is a block of code, marked by a **line label**, that runs when a runtime error occurs. It is enabled with `On Error GoTo label` and lets the program show a friendly message, fix the problem and continue instead of crashing.

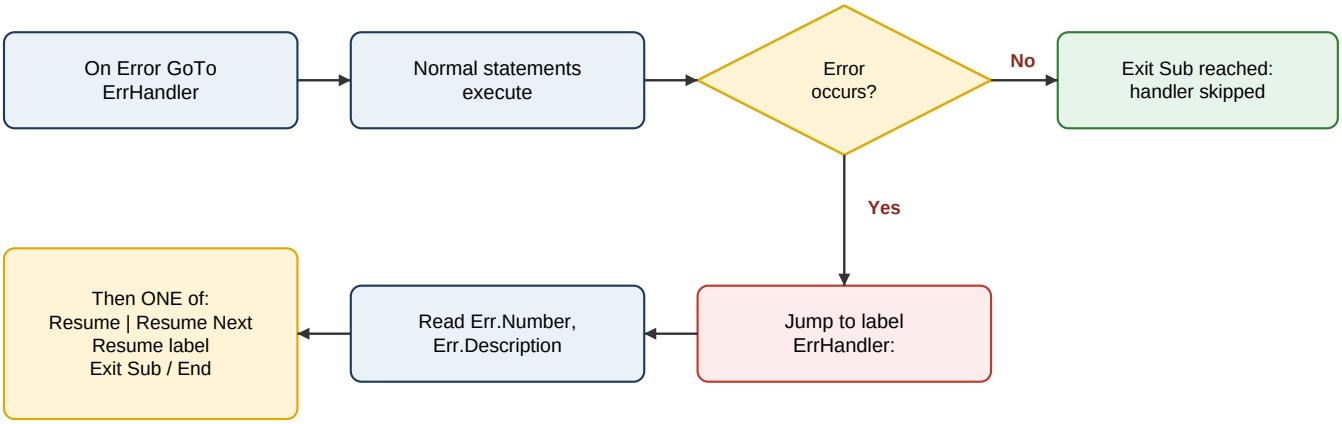

Fig. 6: Error handler execution flow

```
Private Sub cmdDivide_Click()
    Dim a As Double, b As Double, c As Double
    On Error GoTo ErrHandler        ' 1. enable handler
    a = Val(txtA.Text)
    b = Val(txtB.Text)
    c = a / b                       ' error 11 if b = 0
    lblResult.Caption = c
    Exit Sub                        ' 2. keeps normal flow out of handler
ErrHandler:                         ' 3. label ends with colon
    Select Case Err.Number
        Case 11
            MsgBox Err.Description, vbExclamation
            b = 1                   ' correct the cause
            Resume                  ' retry the failing statement
        Case Else
            MsgBox "Error " & Err.Number & ": " & Err.Description
            Exit Sub
    End Select
End Sub
```

OUTPUT

Test 1: txtA = 10, txtB = 2

```
lblResult.Caption shows:  5          (no message box)
```

Test 2: txtA = 10, txtB = 0

```
Message box (exclamation icon):  Division by zero    [OK]
After OK: b is set to 1, Resume retries c = a / b,
lblResult.Caption shows:  10
```

Err object member	Meaning
Number	Error code (0 = no error)
Description	Text message of the error
Source	Name of the object / application that raised the error
HelpFile, HelpContext	Help file and topic ID for the error
LastDllError	Error code from the last DLL call (read-only)
Clear (method)	Resets all Err properties
Raise (method)	Generates an error on purpose: `Err.Raise 5`

Statement ending a handler	Effect
Resume	Re-executes the statement that caused the error
Resume Next	Continues with the statement after the one that caused the error
Resume label	Continues at the given label
Exit Sub / Function / Property	Leaves the procedure; the handler ends
Err.Raise / End	Passes the error on to the caller / stops the program

IMPORTANT FOR OAVS EXAM

- Syntax: `On Error GoTo label`. A handler is part of the procedure - it is **not** a separate Sub or Function.
- **Exit Sub** (or Exit Function) must be placed **before** the handler label.
- If the error occurred in a called procedure, **Resume** re-executes the statement in the handler's own procedure that made the call.
- `Error$(n)` returns the message for error number n; `Erl` returns the line number (only if lines are numbered).

COMMON MISTAKES / CONFUSING POINTS

- Forgetting **Exit Sub**: normal execution falls into the handler even without an error.
- **Resume** without fixing the cause creates an **endless loop**.
- An error raised *inside* the handler is not handled by the same handler - it goes to the caller or the default handler.
- **Resume** vs **Resume Next**: Resume retries the same statement; Resume Next skips it.

SUMMARY

On Error GoTo label enables the handler; Exit Sub guards it; inspect Err; finish with Resume, Resume Next, Resume label or Exit.

D. VB ERROR HANDLING OPTIONS

D (i) How VB handles the runtime error

*(This syllabus title also appears in C; C (i) explained the default flow. Here the **options** that control that behaviour are covered.)*
Definition & purpose. VB gives two sets of options: the **Error Trapping** setting of the IDE, which decides when VB enters Break mode during development, and the **On Error** statement forms, which decide what the program does when an error occurs.

IDE Error Trapping (Tools > Options > General)	Behaviour
Break on All Errors	Stops at every error, even if a handler exists (handler is bypassed) - best for locating the exact line
Break in Class Module	Stops on an unhandled error inside a class module, at the line in the class that raised it
Break on Unhandled Errors (default)	Stops only if no handler is active; handled errors go to the handler

On Error form	Effect
On Error GoTo *label*	Enables the handler starting at the label (see C ii)
On Error Resume Next	Ignore the error and continue with the next statement (used for inline handling, E)
On Error GoTo 0	Disables the enabled handler in the current procedure (see D ii)

IMPORTANT FOR OAVS EXAM

- Three Error Trapping options: **Break on All Errors, Break in Class Module, Break on Unhandled Errors** - the last is the default.
- Error Trapping applies only inside the VB IDE; a compiled EXE always follows its On Error statements.

COMMON MISTAKES / CONFUSING POINTS

- **Break on All Errors** seems to 'ignore' your handler while debugging - the handler works again once the setting is changed back.
- The Error Trapping option is a *development* setting; On Error is *program code*.

SUMMARY

Error trapping option = when the IDE breaks; On Error GoTo / Resume Next / GoTo 0 = how the program reacts.

D (ii) Disabling the error handler

Definition & purpose. `On Error GoTo 0` turns off the error handler that is currently enabled in the procedure. Errors after it are treated as **unhandled** (default VB handling or the caller's handler). It is used to protect only a specific block of code, or to see the real error while debugging.

```
Sub ReadData()
    On Error GoTo Fail            ' handler ON
    Open "C:\data.txt" For Input As #1
    On Error GoTo 0               ' handler OFF from here
    ' ... later errors are NOT trapped ...
    Exit Sub
Fail:
    MsgBox "File problem: " & Err.Description
End Sub
```

OUTPUT

Case 1: C:\data.txt exists

```
No message. The file opens, the handler is switched off and the
remaining code runs.
```

Case 2: C:\data.txt is missing (error 53)

```
Message box:  File problem: File not found    [OK]
The procedure then ends; the code after On Error GoTo 0 never runs.
```

Case 3: a later line causes an error (handler is OFF)

```
VB shows the default dialog:  Run-time error '11': Division by zero
[Continue] [End] [Debug] [Help]   (a compiled EXE terminates)
```

IMPORTANT FOR OAVS EXAM

- **On Error GoTo 0** is the statement that disables error handling (also cancels On Error Resume Next).
- A handler is also disabled automatically when its procedure ends.

COMMON MISTAKES / CONFUSING POINTS

- **GoTo 0 does not mean 'jump to line 0'** - it switches the handler off.
- Do not confuse disabling a handler (GoTo 0) with ignoring errors (Resume Next) or with clearing Err (Err.Clear).

SUMMARY

On Error GoTo 0 = handler off; later errors are unhandled.

E. INLINE ERROR HANDLING

Definition & purpose. In **inline error handling** the programmer uses `On Error Resume Next` and tests `Err.Number` **immediately after a risky statement** instead of jumping to a handler label. It keeps error handling next to the code that may fail.

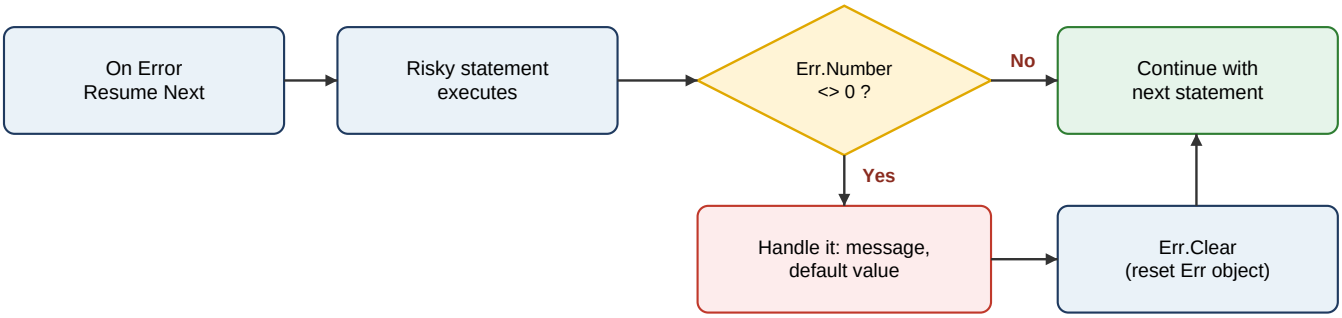

Fig. 7: Inline error handling flow

```
Dim n As Integer
On Error Resume Next            ' errors no longer stop the program
n = CInt(txtAge.Text)           ' risky statement
If Err.Number <> 0 Then         ' test at once
    MsgBox "Enter a valid number: " & Err.Description
    Err.Clear                   ' reset Err object
    txtAge.SetFocus
    Exit Sub
End If
On Error GoTo 0                 ' restore normal handling
```

OUTPUT

Test 1: txtAge = 25

```
No message. n = 25, Err.Number = 0 and the procedure continues.
```

Test 2: txtAge = abc

```
Message box:  Enter a valid number: Type mismatch    [OK]
Err is cleared, the cursor returns to txtAge and the procedure exits.
```

Inline vs Handler - Comparison

Point	Inline (On Error Resume Next)	Handler (On Error GoTo label)
Control flow	No jump; execution continues	Jumps to a label
Where errors are checked	After each risky statement	In one block at end of procedure
Best for	Few specific statements (open file, convert value)	Whole procedure protection
Risk	Errors ignored silently if Err is not tested	Endless loop with careless Resume

IMPORTANT FOR OAVS EXAM

- Inline handling = **On Error Resume Next + If Err.Number <> 0**.
- Err is reset automatically by Resume, Exit Sub/Function/Property and On Error statements; otherwise call **Err.Clear** after handling.
- Use **On Error GoTo 0** afterwards to switch off the 'ignore errors' mode.

COMMON MISTAKES / CONFUSING POINTS

- **On Error Resume Next** (a statement that enables ignoring) is different from **Resume Next** (used inside a handler).
- Putting On Error Resume Next at the top and never testing Err hides bugs.
- Err.Number stays set until cleared, so a later check may report an old error.

SUMMARY

Inline: Resume Next, test Err.Number straight away, handle, Err.Clear, continue.

QUICK REVISION

Key definitions

- **Bug / error:** fault causing failure or wrong result; **debugging:** finding and removing errors.
- **Syntax error:** grammar mistake found before running; **runtime error:** failure during execution; **logical error:** wrong result with no message.
- **Break mode:** paused state of a running program in which it can be examined.
- **Watch expression:** expression monitored by VB; may break when True or when changed.
- **Error handler:** labelled code block enabled by On Error GoTo label.
- **Inline error handling:** On Error Resume Next + checking Err.Number after the risky statement.

Debugging tools at a glance

Tool	Open / use	Main use
Immediate window	Ctrl+G; ? ; Debug.Print	Print values, run statements, change variables
Locals window	View > Locals Window	Auto list of current-procedure variables (Expression, Value, Type)
Watch window	Debug > Add Watch; View > Watch Window	Monitor expressions; break on True / change
Quick Watch	Shift+F9	Temporary value of an expression
Breakpoint	F9	Pause at a chosen line
Stepping	F8, Shift+F8, Ctrl+Shift+F8, Ctrl+F8	Step Into, Step Over, Step Out, Run To Cursor

Error handling concepts

Statement / object	Meaning
On Error GoTo label	Enable handler; place Exit Sub before the label
On Error Resume Next	Ignore error, continue; test Err.Number (inline)
On Error GoTo 0	Disable the handler
Resume / Resume Next / Resume label	Retry statement / skip it / continue at label
Err.Number, Description, Source, Clear, Raise	Information about, resetting and generating errors

Important points

- Err.Number: 11 Division by zero, 13 Type mismatch, 9 Subscript out of range, 53 File not found. Breakpoints and watches are not saved; Debug.Print is ignored in the compiled EXE.
- Unhandled runtime error: VB searches the call stack, then shows the default dialog (End / Debug / Help); EXE terminates.
- Resume without fixing the cause = endless loop; missing Exit Sub = handler runs without error.
- On Error GoTo 0 disables handling - it does not jump to line 0.

Likely exam questions

Question	Short answer
Name the three types of errors.	Syntax (compile), Runtime, Logical.
Shortcut of Immediate window?	Ctrl+G.
Which key toggles a breakpoint? Step Into?	F9; F8.
Difference: Step Over vs Step Out.	Step Over runs a called procedure as one step; Step Out finishes the current procedure.
Name the three watch types.	Watch Expression, Break When Value Is True, Break When Value Changes.
Which window lists variables of the current procedure?	Locals window.
Statement to enable an error handler?	On Error GoTo label.
Difference: Resume and Resume Next.	Resume retries the failing statement; Resume Next continues with the following one.
Statement to disable the error handler?	On Error GoTo 0.
What is inline error handling?	On Error Resume Next followed by testing Err.Number after the statement.
Default Error Trapping option?	Break on Unhandled Errors.